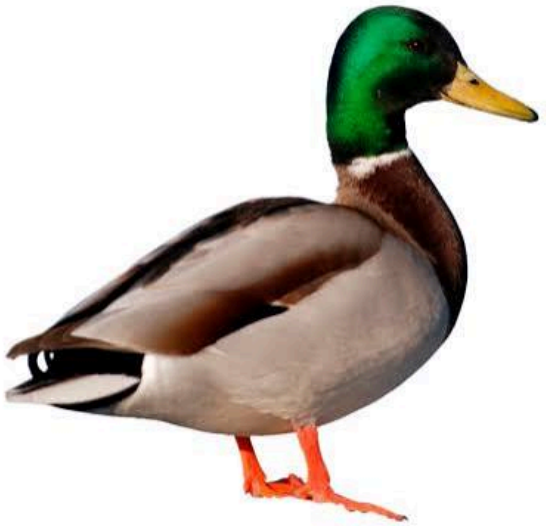
Ciência de Dados Aplicada ao Direito I

Aula 17

Funções em lote e classes



Prof. Bruno da Cunha de Oliveira - 2026/2

Atendimento desta semana: segunda, 19/10

O horário de atendimento desta semana passa de quinta para segunda, 19/10, das 12h30 às 14h.

A sala e a sugestão de exercícios para esse atendimento vão no Blackboard.

Para terça, 20/10: conta no GitHub e Git instalado

Na terça vamos usar o Git e o GitHub. Cheguem com duas coisas feitas:

#	O que fazer
1	Instalar o Git no seu computador
2	Criar uma conta em `github.com`

O guia está no Blackboard, em versão para Windows e para Mac: são os passos 1 e 2. Quem travar faz comigo no começo da aula.

A função dentro do for

Insper

Chamar a função dentro do for

```
def prazo_em_dobro(dias):
    return dias * 2

prazos = [15, 5]
dobrados = []
for simples in prazos:
    dobrado = prazo_em_dobro(simples)
    dobrados.append(dobrado)
print(dobrados)
```

A parte tem prazo em dobro em dois prazos. O que aparece na tela?

A função dentro do for, passo a passo

Animação no navegador: `animacao-lote.html`, cenário `prazos`.

Uma chamada por volta

RESPOSTA

```
[30, 10]
```

A linha da chamada está escrita uma vez, e o `for` a repete. Na primeira volta, `dias` recebe 15; na segunda, 5. Aqui o `return` devolve a conta direto, sem guardar o resultado numa variável antes.

O mesmo resultado sem função

```
prazos = [15, 5]
dobrados = []
for simples in prazos:
    dobrados.append(simples * 2)
print(dobrados)
```

Este programa também mostra `[30, 10]`, e não tem função. Em que situação a versão com função passa a valer a pena?

Quando a função vale a pena

RESPOSTA

Com a conta numa linha e num lugar só, tanto faz.

A função vale quando	O que ela resolve
a regra aparece em mais de um lugar	mudou a regra, corrige-se num lugar só
a regra tem `if` e `else` dentro	o `for` continua curto e legível
a regra precisa ser conferida	testa-se um caso antes de rodar em mil

A triagem de dois casos de estelionato

```
def configura_estelionato(vantagem, prejuizo, erro, fraude):
    return vantagem and prejuizo and erro and fraude

casos = [{"investigado": "Marina Alves", "vantagem": True,
          "prejuizo": True, "erro": True, "fraude": False},
         {"investigado": "Otávio Bastos", "vantagem": True,
          "prejuizo": True, "erro": True, "fraude": True}]
```

Escrevam o `for` que aplica a função a cada caso e guarda as respostas na lista `conclusoes`.

Os campos do caso viram os argumentos

RESPOSTA

```
conclusoes = []
for caso in casos:
    conclusao = configura_estelionato(
        caso["vantagem"], caso["prejuizo"],
        caso["erro"], caso["fraude"])
    conclusoes.append(conclusao)
```

`conclusoes` fica com `[False, True]`. A função é a mesma de um caso só, e quem repete é o `for`.

O for dentro da função

Uma função para a pauta inteira

```
def configura_estelionato(vantagem, prejuizo, erro, fraude):
    return vantagem and prejuizo and erro and fraude
```

Amanhã chega outra pauta, e o `for` da triagem teria de ser escrito de novo. Escrevam a função `triar`: ela recebe a lista de casos e devolve a lista das conclusões, na mesma ordem.

Ela deve chamar `configura_estelionato` para cada caso.

A função da pauta chama a função do caso

RESPOSTA

```
def triar(casos):
    conclusoes = []
    for caso in casos:
        conclusoes.append(configura_estelionato(
            caso["vantagem"], caso["prejuizo"],
            caso["erro"], caso["fraude"]))
    return conclusoes
```

`triar(casos)` devolve `[False, True]`. O `return` fica fora do `for`.

Uma função que escreve o registro

Registrar a conclusão de um caso

```
modelo = "<INVESTIGADO>: configura estelionato? <CONCLUSAO>"

def registrar(modelo, investigado, conclusao):
    modelo.replace("<INVESTIGADO>", investigado)
    modelo.replace("<CONCLUSAO>", str(conclusao))
    return modelo

print(registrar(modelo, "Otávio Bastos", True))
```

A lista `[False, True]` não diz de quem é cada resposta. Esta função escreve o registro de um caso. O que aparece na tela?

Guardar o que o replace devolve

RESPOSTA

```
<INVESTIGADO>: configura estelionato? <CONCLUSAO>
```

`.replace()` devolve um texto novo e não altera `modelo`.

```
    texto = modelo.replace("<INVESTIGADO>", investigado)
    texto = texto.replace("<CONCLUSAO>", str(conclusao))
    return texto
```

A segunda troca parte de `texto`, e o `str()` passa o `True` para texto.

Desafio V, no PrairieLearn: para entregar até 21/10

Vale 0,1 ponto extra na prova final para quem concluir até o início da aula de quarta, 21/10, com todas as questões aceitas pelo corretor. Dá para tirar dúvida no horário de atendimento.

Insper

A ordem dos argumentos

Insper

Ler uma chamada com quatro argumentos

```
def configura_estelionato(vantagem, prejuizo, erro, fraude):
    return vantagem and prejuizo and erro and fraude

resposta = configura_estelionato(True, True, False, True)
```

Qual dos quatro fatos não foi apurado neste caso? Como vocês descobriram?

Argumento nomeado

RESPOSTA

Foi `erro`, o terceiro. Para saber, é preciso contar a posição e voltar à definição.

```
resposta = configura_estelionato(
    vantagem=True, prejuizo=True,
    erro=False, fraude=True)
```

Cada argumento pode ir com o nome do parâmetro. A chamada passa a dizer os fatos.

Um programa de verdade

range(1, 4) num for

```
for pagina in range(1, 4):
    print(pagina)
```

O que aparece na tela?

range: até antes do segundo número

RESPOSTA

```
1
2
3
```

`range(1, 4)` entrega 1, 2 e 3: começa no primeiro número e para antes do segundo. Para ir da página 1 à 10, escreve-se `range(1, 11)`.

Um programa com o juscraper

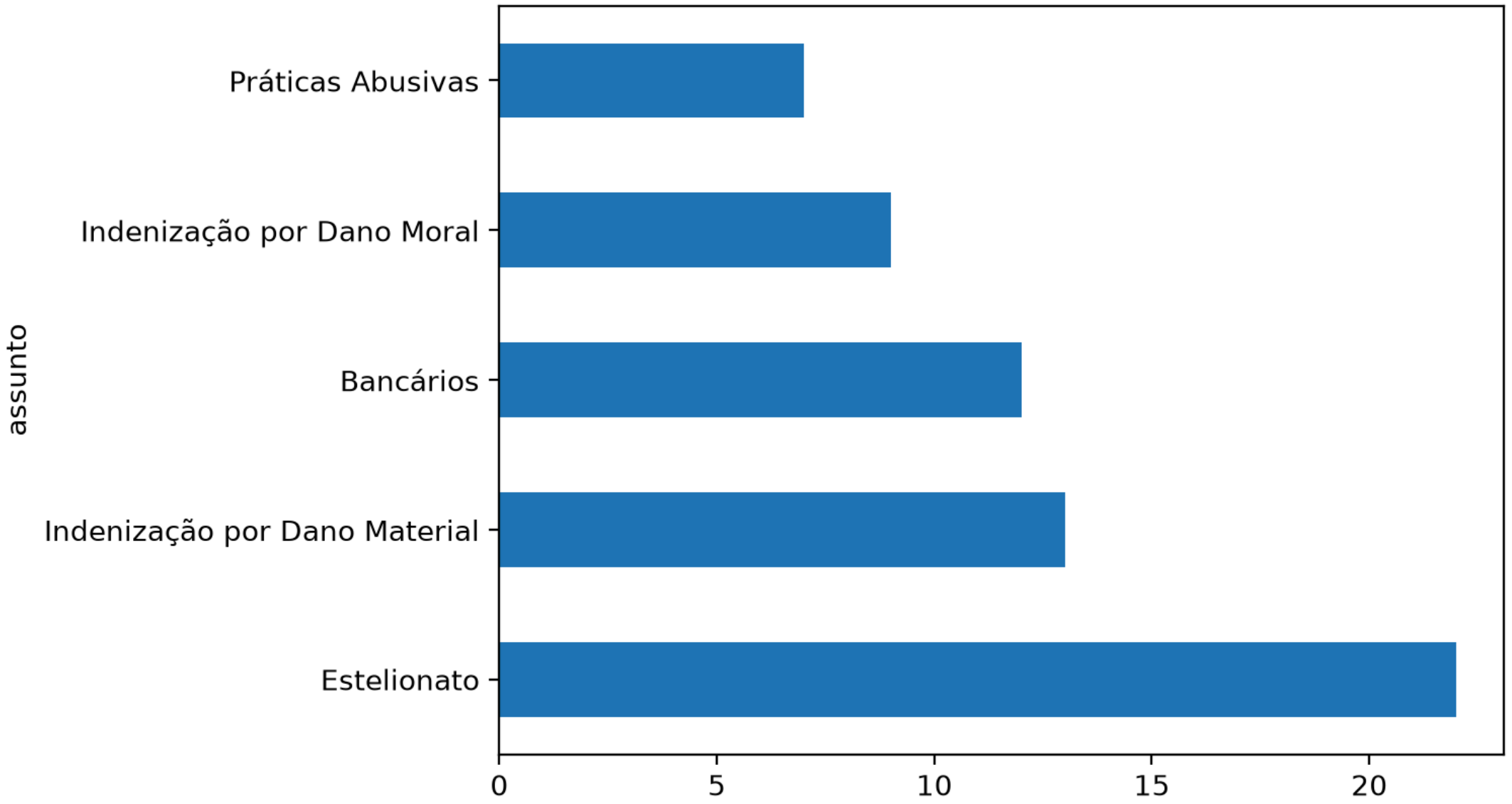
Leiam sozinhos. Que partes vocês já entendem, e que partes ainda são mistério?

```
import juscraper as jus

tjsp = jus.scraper("tjsp")
dados = tjsp.cjpg("golpe do pix", paginas=range(1, 11))

dados.shape
contagem = dados["assunto"].value_counts()
contagem.head(5).plot(kind="barh")
```

O gráfico que esse programa produz




SONY
Discman
GAME BOY COLOR
TYCO
TYCO
27MHz


Oct 28, Oct 28
10:09

Python não vem com pilha

O que vem e o que não vem com o Python

VEM COM O PYTHON

Texto, número, lista e dicionário. `if`, `for` e funções.

É tudo o que vocês usaram até agora.

O que vem e o que não vem com o Python

VEM COM O PYTHON

Texto, número, lista e dicionário. `if`, `for` e funções.

É tudo o que vocês usaram até agora.

NÃO VEM

Tabelas. Gráficos. Leitura de PDF. Coleta de decisões em tribunais. Modelos de linguagem.

É quase tudo o que um trabalho com dados precisa.

Bibliotecas

O que não vem com o Python se instala.

São as bibliotecas: código que outra pessoa escreveu e publicou. O juscraper é uma delas.

Entendendo o código de bibliotecas

Insper

De bibliotecas a objetos

1 Para fazer coisas mais interessantes, usamos bibliotecas. Ninguém reescreve, a cada trabalho, um coletor de decisões de tribunal ou uma tabela de dados.

De bibliotecas a objetos

1. Para fazer coisas mais interessantes, usamos bibliotecas. Ninguém reescreve, a cada trabalho, um coletor de decisões de tribunal ou uma tabela de dados.

2. Para entender como uma biblioteca funciona, precisamos entender objetos. É em objetos que quase toda biblioteca entrega o que faz: o coletor é um objeto, a tabela é um objeto.

De bibliotecas a objetos

1. Para fazer coisas mais interessantes, usamos bibliotecas. Ninguém reescreve, a cada trabalho, um coletor de decisões de tribunal ou uma tabela de dados.

2. Para entender como uma biblioteca funciona, precisamos entender objetos. É em objetos que quase toda biblioteca entrega o que faz: o coletor é um objeto, a tabela é um objeto.

3. Saber que `dados.shape` guarda um dado e que `contagem.head(5)` executa uma ação é o que permite ler a documentação de uma biblioteca, a mensagem de erro e o código que a IA escreve.

O ponto depois de um texto

```
tribunal = "TJSP"
print(tribunal.lower())
print(tribunal)
```

O que aparece na tela?

Um método: o texto sabe virar minúsculas

RESPOSTA

```
tjsp
TJSP
```

O ponto pede uma ação ao valor que vem antes dele: `tribunal.lower()` devolve o texto em minúsculas. Como o `.replace()`, ele devolve um texto novo, e `tribunal` continua como estava. Essa ação se chama método.

Métodos de texto

Chamada	Devolve	O que faz
`"TJSP".lower()`	`tjsp`	passa para minúsculas
`"tjsp".upper()`	`TJSP`	passa para maiúsculas
`"  TJSP  ".strip()`	`TJSP`	tira os espaços das pontas
`"TJSP".replace("SP", "RJ")`	`TJRJ`	troca um trecho por outro

Vocês já usavam métodos: `.replace()` é do texto, e `.append()`, da lista.

O ponto depois de uma lista

```
pecas = ["petição inicial", "contestação"]
pecas.append("sentença")
print(pecas)
print(pecas.lower())
```

O que aparece na tela?

Cada tipo tem os seus métodos

RESPOSTA

```
['petição inicial', 'contestação', 'sentença']
AttributeError: 'list' object has no attribute 'lower'
```

`.append()` é método de lista, e `.lower()`, de texto. Uma lista não sabe virar minúsculas, e o Python diz isso no erro: a lista não tem `lower`.

Função ou método

Elemento	Como se usa	Exemplos
função	vive solta, e o valor entra entre os parênteses	`len(pecas)`, `print(pecas)`
método	mora dentro de um tipo, e age sobre o valor antes do ponto	`pecas.append("sentença")`, `tribunal.lower()`

Um método é uma função que já sabe sobre qual valor vai agir.

Criar um tipo

Insper

O molde e os cachorros

A CLASSE É O MOLDE

```
CLASSE Cachorro:
    nome
    idade
    latir()
```

CADA CACHORRO É UM OBJETO

```
rex:  nome "Rex",  idade 3
toto: nome "Totó", idade 8
```

O molde é escrito uma vez. Cada cachorro sai dele com os seus valores. Texto e lista são tipos que alguém escreveu assim, e dá para escrever os nossos.

class e `__init__`

```python
class Cachorro:
    def __init__(self, nome, idade):
```

`class` cria o tipo e dá o nome a ele: é o molde.

`__init__` é uma função como as que vocês já escrevem, com um nome reservado. O Python a chama sozinho, toda vez que um cachorro é criado: é ela que preenche a ficha dele.

self: este cachorro

```
class Cachorro:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade
```

`self` é o cachorro que está sendo criado.

`self.nome = nome` guarda neste cachorro o nome que chegou pela chamada.
`self.idade = idade` faz o mesmo com a idade.

Criar dois cachorros

```
class Cachorro:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

rex = Cachorro("Rex", 3)
toto = Cachorro("Totó", 8)
print(rex.idade, toto.idade)
```

O que aparece na tela?

Atributo: o que cada objeto guarda

RESPOSTA

```
3 8
```

`Cachorro("Rex", 3)` chama o `__init__`: `"Rex"` vai para `nome`, e 3, para `idade`. Quem chama não escreve o `self`. `rex.idade` lê o que ficou guardado no Rex: é um atributo, e cada cachorro tem os seus.

Um cachorro que late

```
class Cachorro:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def latir(self):
        return self.nome + ": au au"
toto = Cachorro("Totó", 8)
print(toto.latir())
```

Acrescentei uma função dentro da classe. O que aparece na tela?

Método: uma função dentro da classe

RESPOSTA

```
Totó: au au
```

Uma função escrita dentro da classe é um método. Em `toto.latir()`, `self` é o `toto`, e `self.nome` vale `"Totó"`. É o mesmo ponto de `tribunal.lower()`: pede uma ação ao objeto.

Atributo e método

```
toto = Cachorro("Totó", 8)
```

Elemento	O que é	Neste caso
atributo	um dado que o objeto guarda; sem parênteses	`toto.idade`
método	uma ação que o objeto executa; com parênteses	`toto.latir()`

Programação orientada a objetos

Os dados e o que se faz com eles andam juntos, num objeto.

Programação orientada a objetos

Os dados e o que se faz com eles andam juntos, num objeto.

COM VARIÁVEIS SOLTAS

`numero`, `tribunal` e `resultado` de duzentos processos são seiscentas variáveis para não misturar. No objeto, cada processo carrega as suas.

Programação orientada a objetos

Os dados e o que se faz com eles andam juntos, num objeto.

COM VARIÁVEIS SOLTAS

`numero`, `tribunal` e `resultado` de duzentos processos são seiscentas variáveis para não misturar. No objeto, cada processo carrega as suas.

COM FUNÇÕES SOLTAS

Quem chama precisa passar tudo, e saber qual função serve para qual dado. O método já sabe onde está: `toto.latir()`.

A ficha da parte

```python
class Parte:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def tem_prioridade(self):
        return self.idade >= 60
```

Na autuação, alguém preenche a ficha de cada parte. A classe é a ficha em branco, o `__init__` é o preenchimento, e cada parte do processo é uma ficha preenchida.

Criar uma parte, passo a passo

Animação no navegador: `animacao-objetos.html`, cenário `criar`.

Escrever a classe Processo

Escrevam a classe `Processo`, com dois atributos e um método.

Os atributos são `numero`, o número do processo, e `valor_sm`, o valor da causa em salários mínimos. O método `dentro_do_teto()` devolve `True` quando o valor não passa de 40 salários mínimos, o teto de valor do Juizado Especial Cível, e `False` quando passa.

Vale 0,1 ponto extra na prova final para quem entregar a classe pelo formulário "classe Processo", no Blackboard, até o fim da aula, com as duas respostas do próximo slide certas.

Conferir antes de entregar

Depois da classe de vocês, rodem estas três linhas:

```
causa = Processo("0000123-27.2024.8.99.0001", 15)
outra = Processo("0000456-76.2024.8.99.0001", 50)
print(causa.dentro_do_teto(), outra.dentro_do_teto())
```

A classe está pronta para entregar quando aparecer:

```
True False
```

A classe Processo

RESPOSTA

```python
class Processo:
    def __init__(self, numero, valor_sm):
        self.numero = numero
        self.valor_sm = valor_sm

    def dentro_do_teto(self):
        return self.valor_sm <= 40
```

A comparação já vale `True` ou `False`, e o `return` a entrega.

O programa do juscraper, de novo

```
import juscraper as jus

tjsp = jus.scraper("tjsp")
dados = tjsp.cjpg("golpe do pix", paginas=range(1, 11))

dados.shape
contagem = dados["assunto"].value_counts()
contagem.head(5).plot(kind="barh")
```

Leiam de novo. O que deixou de ser mistério?

O que já dá para ler

Animação no navegador: `animacao-leitura-juscraper.html`, cenário `primeira-volta`.